

floatScaleThreeHalves: 51/60 operators

floatRoundEven: 38/65 operators

float_i2f: 26/40 operators

bitCount: 36/40 operators

bitReverse: 28/34 operators

All 19 functions passed operator checks.

operator check passed

Score	Rating	Errors	Function
1	1	0	signMask
2	2	0	bitXor
3	3	0	negativePart
4	4	0	copyByteWithin
4	4	0	logicalShift
4	4	0	swapNibblePairs
4	4	0	secondLowestZeroBit
5	5	0	oddParity
5	5	0	rotateRightBits
5	5	0	roundEvenPow2
5	5	0	midpointTowardFirst
7	7	0	isBetweenEitherOrder
7	7	0	mul5Sat
7	7	0	classifyAdd3
7	7	0	floatScaleThreeHalves
10	10	0	floatRoundEven
10	10	0	float_i2f
10	10	0	bitCount
10	10	0	bitReverse

Total points: 110/110

all checks passed: 110/110

xiu@LAPTOP-6Q0089OK:~/Lab1\$

7	7	0	classifyAdd3
7	7	0	floatScaleThreeHalves
10	10	0	floatRoundEven
10	10	0	float_i2f
10	10	0	bitCount
10	10	0	bitReverse

Total points: 110/110

all checks passed: 110/110

xiu@LAPTOP-6Q0089OK:~/Lab1\$./check_ops.py bits.c

signMask: 1/2 operators

bitXor: 7/8 operators

negativePart: 4/6 operators

copyByteWithin: 10/12 operators

logicalShift: 6/20 operators

swapNibblePairs: 9/18 operators

secondLowestZeroBit: 8/8 operators

oddParity: 12/56 operators

rotateRightBits: 11/16 operators

roundEvenPow2: 10/24 operators

midpointTowardFirst: 21/32 operators

isBetweenEitherOrder: 47/48 operators

mul5Sat: 23/30 operators

classifyAdd3: 39/52 operators

floatScaleThreeHalves: 51/60 operators

floatRoundEven: 38/65 operators

float_i2f: 26/40 operators

bitCount: 36/40 operators

bitReverse: 28/34 operators

All 19 functions passed operator checks.

xiu@LAPTOP-6Q0089OK:~/Lab1\$ |

```
classifyAdd3: 39/52 operators
floatScaleThreeHalves: 51/60 operators
floatRoundEven: 38/65 operators
float_i2f: 26/40 operators
bitCount: 36/40 operators
bitReverse: 28/34 operators
All 19 functions passed operator checks.
```

```
xiu@LAPTOP-6Q0089OK:~/Lab1$ ./btest
```

Score	Rating	Errors	Function
1	1	0	signMask
2	2	0	bitXor
3	3	0	negativePart
4	4	0	copyByteWithin
4	4	0	logicalShift
4	4	0	swapNibblePairs
4	4	0	secondLowestZeroBit
5	5	0	oddParity
5	5	0	rotateRightBits
5	5	0	roundEvenPow2
5	5	0	midpointTowardFirst
7	7	0	isBetweenEitherOrder
7	7	0	mul5Sat
7	7	0	classifyAdd3
7	7	0	floatScaleThreeHalves
10	10	0	floatRoundEven
10	10	0	float_i2f
10	10	0	bitCount
10	10	0	bitReverse

```
Total points: 110/110
```

```
xiu@LAPTOP-6Q0089OK:~/Lab1$ |
```

P1 signMask

• 思路：生成最高位为1的掩码。由于常数限制，不能直接写 0x80000000。利用移位操作，将常数 1 左移 31 位，得到最高位为 1 其余为 0 的掩码。

P2 bitXor

• 思路：仅使用 ~ 和 & 实现异或。根据德摩根定律，异或逻辑可表示为 $x \oplus y = \sim(\sim x \& \sim y) \& \sim(x \& y)$ ，即“至少有一个为1，且不同时为1”。将两部分按位与，完美实现异或并满足操作符限制。

P3 negativePart

• 思路：求负数绝对值的条件表达式。利用算术右移 $x \gg 31$ 作为掩码：若x为负，掩码全1；若x为正，掩码全0。将掩码与 $\sim x + 1$ （即 $-x$ ）进行与运算，即可实现正数返回0，负数返回相反数的效果。

P4 copyByteWithin

• 思路：字节复制操作。分为三步：1. 将源字节移动到最低8位并保留；2. 将其左移到目标字节位置；3. 通过掩码 $\sim(0xFF \ll (dst \ll 3))$ 将原目标字节清零，最后将两部分进行或运算拼接。

P5 logicalShift

• 思路：实现逻辑右移。由于C语言右移是算术右移，高位会补符号位。我们先提取 $1 \ll 31$ ，通过构造 $\sim(((1 \ll 31) \gg n) \ll 1)$ 生成高n位为0、其余为1的掩码，再与算术右移的结果相与，清除多余的符号扩展位。

P6 swapNibblePairs

• 思路：交换字节内的高低4位。构造 0x0F0F0F0F 掩码，将原数分成高4位和低4位两部分。低4位左移4位，高4位右移4位，两者进行或运算重新组合，实现半字节互换。

P7 secondLowestZeroBit

• 思路：寻找第二低位0。对原数取反 $y = \sim x$ ，将问题转化为找第二低位的1。先用 $lowest_1 = y \& (\sim y + 1)$ 提取最低位1，再用异或 $y \oplus lowest_1$ 将其剔除，最后对新的数再次使用提取公式，得到第二低位的1。

P8 oddParity

• 思路：奇偶校验。利用异或运算“相异为1，相同为0”的特性，采用分治法。将32位数依次右移16位、8位、4位、2位、1位并进行异或折叠。最终折叠到最低位，若为1说明1的个数为奇数，0则为偶数。最后取反以满足题目“偶数返回1”的要求。

P9 rotateRightBits

• 思路：循环右移。需要将低n位移动到高。先用掩码消除算术右移的符号扩展得到逻辑右移部分，然后将原数左移 $32-n$ 位得到循环的高位部分（补码求相反数避免减法）。两部分按位或实现完整循环右移。

P10 roundEvenPow2

• 思路：向偶数舍入到 2^n 的倍数。核心是构造偏置量。基础偏置为 $2^{(n-1)}-1$ 。若目标数为奇数，我们需要额外加1让其进位为偶数。计算偏置后，加到原数上，先右移n位舍去余数，再左移n位还原倍数，实现无分支的向偶数舍入。

P11 midpointTowardFirst

• 思路：无溢出求平均值并偏向第一个参数。使用 $(x \& y) + ((x \oplus y) \gg 1)$ 安全计算向下取整的平均值。随后判断是否需要+1调整：条件为x和y之和为奇数 $((x \oplus y) \& 1)$ ，且x大于y（通过符号位和差值符号位组合安全判断，避免减法溢出）。

P12 isBetweenEitherOrder

• 思路：判断x是否在a和b之间。难点在于无溢出比较大小。利用防溢出比较公式 $(diff \oplus ((sa \oplus sb) \& (diff \oplus a))) \gg 31$ 得到 $a < b$ 的掩码。提取最小值min和最大值max。随后用无溢出比较法分别判断 $min \leq x$ 和 $x \leq max$ 。最后合并结果并强制返回0或1。

P13 mul5Sat

• 思路：饱和乘5。为防溢出，拆解为 $x + (x \ll 2)$ 。核心在于“全链路符号监控”：计算 $2x$ 、 $4x$ 、 $5x$ 每一步的符号位，并与x的符号位异或，检测任一步骤是否发生回绕溢出。若发生溢出，根据x的符号返回INT_MAX（利用 $\sim initmin$ 构造）或INT_MIN。否则直接返回结果。

P14 classifyAdd3

• 思路：三数和溢出分类。分两步相加： $s1 = x + y$ ， $s2 = s1 + z$ 。分别检测第一步和第二步的正负溢出。关键在于：若第一步发生正溢出而第二步发生负溢出（或反之），说明溢出相互抵消，最终结果并未溢出。通过掩码剔除相互抵消的情况，合并得到最终的 pos_final 和 neg_final 并返回对应值。

P15 floatScaleThreeHalves

• 思路：浮点数乘1.5。提取符号位、阶码、尾数。对于规格化数，补上隐藏位1，计算 $m3 = m + (m \ll 1)$ 即乘以3。然后除以2（右移）。舍入时采用“向偶数舍入”：提取余数，若余数大于一半，或等于一半且保留位为奇数，则进位。最后处理阶码进位和特殊值（Inf/NaN，非规格化数，零）。

P16 floatRoundEven

• 思路：浮点数舍入到最近整数。分情况讨论指数大小：若指数 < 126 ，直接舍入到0（保留符号）；若等于126，根据尾数判断舍入到0或1.0；若在 $[1, 2^{23})$ 之间，提取整数部分和小数部分，根据“向偶数舍入”规则（小数 > 0.5 或 小数 $= 0.5$ 且整数部分为奇数时进位）调整整数部分；若 $\geq 2^{23}$ ，本身就是整数直接返回。最后将整数结果重新组装为浮点数。

P17 float_i2f

• 思路：整数转浮点数。处理0和符号位。利用循环寻找最高有效位（MSB）确定阶码。根据有效位数是否超过23位，决定截断或左移。若需截断，提取被丢弃的位用于“向偶数舍入”判断。舍入后若发生进位导致尾数溢出，需增加阶码并重置尾数。最后按IEEE 754格式拼接结果。

P18 bitCount

• 思路：并行位计数（SWAR算法）。核心是分治归并：先每2位统计1的个数，再每4位、8位、16位、32位合并。为了避免大常数限制，通过移位和或运算拼凑出 0x55555555 等掩码。依次将相邻分组相加，最终将所有位的1的个数累加至最低8位，无循环实现0(1)计数。

P19 bitReverse

• 思路：位反转。采用分治交换法：依次交换相邻1位、2位、4位、8位、16位。构造掩码时，为满足常数限制，利用掩码复用技巧，先构造 $m8$ ，再通过 $m8 \oplus (m8 \ll 4)$ 依次推导 $m4$ 、 $m2$ 、 $m1$ 。最后一步16位交换时，用 $m16$ 掩码屏蔽算术右移带来的符号扩展，确保高16位正确补0。